

Table of Contents

1. Introduction to Selenium WebDriver	2
2. Web Application Overview	2
3. Selenium WebDriver Setup	3
Installation	3
Browser Driver Configuration	3
Opening the Application	3
4. Automated Test Cases.....	3
5. Test Execution	4
6. Problems Faced and Solutions	5
GitHub Repository	5

Selenium WebDriver Testing Report

Bank Account Simulator

1. Introduction to Selenium WebDriver

Selenium WebDriver is an open-source automation framework used to test web applications by simulating real user interactions in a browser. Unlike manual testing where a human opens a browser and performs actions, Selenium does this programmatically. It can type into fields, click buttons, read text from the page, and verify results automatically.

In a DevOps pipeline, automated testing is a critical phase. Every time a developer pushes new code, Selenium tests run automatically to ensure nothing is broken. This makes it an essential tool in Continuous Integration and Continuous Delivery (CI/CD) workflows. Selenium supports multiple programming languages including Python, Java, JavaScript, and C#, and works with all major browsers like Chrome, Firefox, and Edge.

For this project, Selenium WebDriver was used with Python to automate the testing of a browser-based *Bank Account Simulator application*.

2. Web Application Overview

The Bank Account Simulator is a client-side web application built using HTML, CSS, and JavaScript. It simulates basic banking operations without requiring any backend or database. The application runs entirely in the browser and maintains state through JavaScript variables during the session.

The application provides the following functionality:

1. A user can deposit funds into their account
2. Can withdraw funds with overdraft protection
3. Can view a timestamped transaction history,
4. Able to reset the account back to zero.

All interactions are validated on the client side, and appropriate success or error messages are displayed to the user in real time.

The interface was designed with clear element IDs (*amount*, *deposit-btn*, *withdraw-btn*, *reset-btn*, *balance*, *message*) to make Selenium test locators straightforward and reliable.

3. Selenium WebDriver Setup

Installation

Selenium was installed using Python's package manager along with webdriver-manager, which automatically handles ChromeDriver installation and version matching.

```
pip install selenium webdriver-manager pytest
```

Browser Driver Configuration

Instead of manually downloading ChromeDriver, webdriver-manager was used to auto-install the correct version matching the installed Chrome browser. This eliminates version mismatch errors which are a common issue in Selenium setup.

```
from selenium import webdriver
from selenium.webdriver.chrome.service import Service
from webdriver_manager.chrome import ChromeDriverManager

driver = webdriver.Chrome(service=Service(ChromeDriverManager().install()))
```

Opening the Application

Since the application is a local HTML file, an absolute file path was constructed using Python's os.path module to ensure cross-platform compatibility on Windows.

```
path = os.path.abspath(os.path.join(os.path.dirname(__file__), '..', 'app',
'index.html'))
driver.get(f"file:///{"path.replace(os.sep, '/')}")
```

4. Automated Test Cases

All test cases were written using Python's built-in *unittest* framework combined with Selenium WebDriver. The test suite consists of 8 test cases organized in a single class *TestBankSimulator*. Each test opens a fresh browser instance via *setUp()* and closes it via *tearDown()* to ensure complete test isolation.

Three helper methods were created to keep the code clean. *enter_amount()* to type a value into the input field, *click()* to interact with buttons, and *get_message()* / *get_balance()* to read output from the page. *time.sleep()* was used between actions to make the browser interaction visible and stable.

- **TC-01: Valid Deposit** A deposit of \$500 was entered and submitted. The test asserts that the balance displays \$500.00 and the message confirms successful deposit.

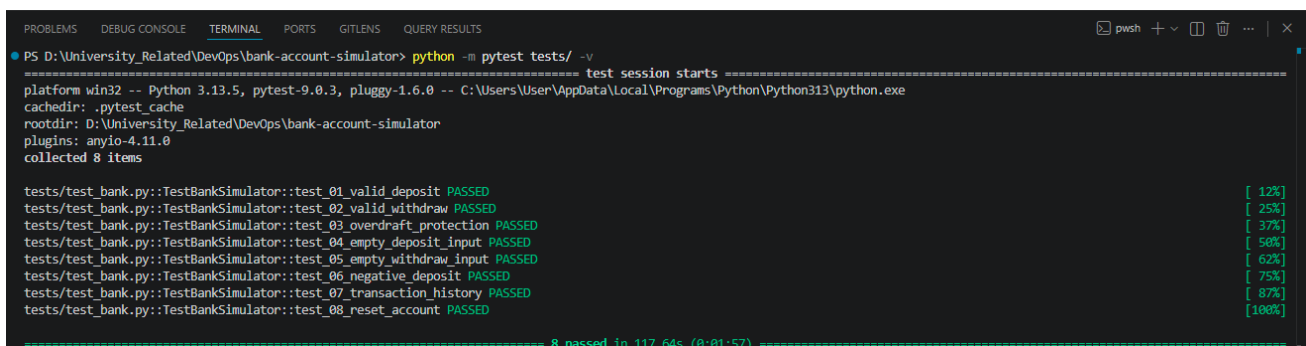
- **TC-02: Valid Withdrawal** After depositing \$500, a withdrawal of \$200 was made. The test asserts the remaining balance shows \$300.00 and the message confirms successful withdrawal.
- **TC-03: Overdraft Protection** With a balance of only \$100, a withdrawal of \$500 was attempted. The test asserts that the error message Insufficient funds is displayed and the balance remains unchanged at \$100.00.
- **TC-04: Empty Deposit Input** The deposit button was clicked without entering any amount. The test asserts the validation message Please enter a valid amount appears.
- **TC-05: Empty Withdrawal Input** Same as TC-04 but for the withdraw button. Asserts the same validation message appears.
- **TC-06: Negative Deposit** A value of -100 was entered as a deposit. The test asserts the message Amount must be positive is displayed, preventing invalid transactions.
- **TC-07: Transaction History Update** After a deposit of \$300 and a withdrawal of \$100, the test verifies that at least 2 rows exist in the transaction history table, confirming it updates correctly.
- **TC-08: Reset Account** After depositing \$500, the reset button was clicked. The test asserts the balance returns to \$0.00 and the confirmation message contains the word reset.

5. Test Execution

All 8 test cases were executed using *pytest* from the project root:

```
python -m pytest tests/ -v
```

All 8 tests passed successfully. During execution, Selenium automatically opened Chrome, navigated to the application, performed interactions, and closed the browser. One test at a time. Screenshots were captured automatically by the test script using *driver.save_screenshot()* and saved into the *screenshots/* directory.



```

PROBLEMS  DEBUG CONSOLE  TERMINAL  PORTS  GIT LENS  QUERY RESULTS
P5 D:\University_Related\DevOps\bank-account-simulator> python -m pytest tests/ -v
===== test session starts =====
platform win32 -- Python 3.13.5, pytest-9.0.3, pluggy-1.6.0 -- C:\Users\User\AppData\Local\Programs\Python\Python313\python.exe
cachedir: .pytest_cache
rootdir: D:\University_Related\DevOps\bank-account-simulator
plugins: anyio-4.11.0
collected 8 items

tests/test_bank.py::TestBankSimulator::test_01_valid_deposit PASSED [ 12%]
tests/test_bank.py::TestBankSimulator::test_02_valid_withdraw PASSED [ 25%]
tests/test_bank.py::TestBankSimulator::test_03_overdraft_protection PASSED [ 37%]
tests/test_bank.py::TestBankSimulator::test_04_empty_deposit_input PASSED [ 50%]
tests/test_bank.py::TestBankSimulator::test_05_empty_withdraw_input PASSED [ 62%]
tests/test_bank.py::TestBankSimulator::test_06_negative_deposit PASSED [ 75%]
tests/test_bank.py::TestBankSimulator::test_07_transaction_history PASSED [ 87%]
tests/test_bank.py::TestBankSimulator::test_08_reset_account PASSED [100%]

===== 8 passed in 117.64s (0:01:57) =====

```

6. Problems Faced and Solutions

ChromeDriver Version Mismatch Initially, manually downloading *ChromeDriver* caused a version mismatch with the installed Chrome browser. This was resolved by using *webdriver-manager* which automatically downloads and manages the correct *ChromeDriver* version.

File Path Issue on Windows Selenium requires a properly formatted file URI to open local HTML files on Windows. Using *os.path.abspath()* combined with replacing backslashes with forward slashes resolved this issue cleanly across all environments.

Tests Running Too Fast By default, Selenium executes actions extremely fast making it difficult to visually observe the browser interactions. Adding *time.sleep()* delays after each action slowed the execution to a human-observable speed, which also helped in capturing meaningful screenshots.

GitHub Repository

<https://github.com/Syed-Muhammad-Hussnain-Raza/bank-account-simulator.git>

